ITA0611 - Machine Learning

Assignment Questions

Name:	KARANAM ASWINI
Reg. :	192312397
Course Code & Name	ITA0611 - Machine Learning
Assignment Title	Design and Development of an Intelligent Early-Warning System for Crop Disease and Yield Risk Prediction Using Machine Learning
Course Outcome(s) - CO	CO6 Instance-based learning including KNN, LWR and case-based reasoning CO7 Data manipulation and analysis using NumPy and Pandas Data cleaning, statistical analysis, aggregation and visualization CO8 Integration and application of theoretical and practical ML knowledge
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 9 - Industry, Innovation and Infrastructure; SDG 13Climate Action

Problem Statement

Farmers often face significant losses due to crop diseases, unfavorable weather conditions, and variations in soil and environmental parameters. Develop an **intelligent machine learning–based early-warning system** that can analyze agricultural data such as temperature, rainfall, humidity, soil moisture, soil nutrients, crop type, disease symptoms, and historical yield to predict **crop disease risk and potential yield loss**. The system should preprocess and analyze the dataset using **NumPy and Pandas**, identify relevant patterns, and implement suitable machine learning approaches such as **Bayesian classification, k-nearest neighbors, locally weighted regression, decision trees, and neural networks**. Students should investigate how different learning approaches represent and search the hypothesis space and compare their predictive performance. A suitable **multilayer perceptron with back propagation** may be developed for risk prediction, while an appropriate **genetic algorithm/genetic programming approach** can be explored for feature or model optimization. The final system should provide interpretable predictions, performance evaluation, visualizations, and recommendations that can support farmers in taking timely preventive or corrective actions.

Question 1 – Data Collection and Preprocessing
Collect a suitable crop-yield and disease dataset containing environmental, soil, crop, and weather attributes. Use **NumPy and Pandas** to clean, preprocess, transform, and analyze the data.
Question 2 – Concept Learning and Version Space
Formulate crop disease-risk prediction as a **concept-learning problem** by defining the hypothesis space, target concept, positive and negative examples, and inductive bias.
Perform **Candidate Elimination/Version Space analysis** using suitable training examples.
Question 3 – Decision Tree Learning
Develop a **Decision Tree classifier** to predict crop disease risk or yield-risk categories.
Use **Information Gain, Gain Ratio, or Gini Index** to select suitable attributes and interpret the resulting decision rules.
Question 4 – Bayesian Classification
Develop a **Bayesian classifier** to predict crop disease risk using environmental, soil, and crop-related attributes. Apply **Bayes' theorem** and estimate the required probabilities using **Maximum Likelihood Estimation (MLE)**.
Question 5 – K-Nearest Neighbors
Implement **K-Nearest Neighbors (KNN)** to classify crop disease or yield-risk levels.
Experiment with different values of *K*, select a suitable value, and analyze its effect on prediction performance.
Question 6 – Locally Weighted Regression

Implement **Locally Weighted Regression (LWR)** to predict crop yield using suitable environmental and soil attributes. Study the effect of the weighting parameter and evaluate the prediction performance.

Question 7 – Neural Network and Back Propagation

Design and implement a **Multilayer Perceptron (MLP)** with **Back Propagation** for crop disease or yield-risk prediction. Experiment with suitable network parameters and evaluate the model's performance.

Question 8 – Genetic Algorithm / Genetic Programming

Apply a **Genetic Algorithm or Genetic Programming** approach for feature selection or machine-learning model optimization.

Define the fitness function and genetic operations, and evaluate the improvement achieved.

Question 9 – Model Evaluation and Comparison

Evaluate the developed models using suitable metrics such as **accuracy, precision, recall, F1-score, MAE, and RMSE**. Compare the models based on performance, interpretability, scalability, and generalization, and identify the most suitable model.

Question 10 – Integrated Crop Risk Early-Warning System

Develop an integrated **Crop Disease and Yield Risk Early-Warning System** using the selected machine-learning approach. Provide predictions, visualizations, and recommendations to help farmers take timely preventive or corrective actions.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Assessment Criteria	**Marks**	**Excellent (4)**	**Good (3)**	**Satisfactory (2)**	**Needs Improvement (1)**
Data Collection, Cleaning & Preprocessing	**10**	Complete and accurate preprocessing with appropriate justification	Mostly correct preprocessing	Basic preprocessing with some omissions	Major preprocessing errors
Statistical Analysis & Visualization	**10**	Comprehensive analysis with meaningful visualizations and interpretations	Good analysis and visualizations	Basic analysis	Limited/inappropriate analysis
Concept Learning, Version Space & Inductive Bias	**10**	Correct hypothesis representation and clear analysis of Version Space/Candidate Elimination	Mostly correct with minor gaps	Partial understanding	Incorrect/incomplete implementation
Decision Tree & Concept Representation	**10**	Correct tree construction, attribute selection and interpretation	Correct implementation with minor issues	Basic implementation	Poor tree construction/interpretation
Bayesian Learning & MLE	**10**	Correct Bayesian formulation, probability calculation and MLE application	Mostly correct implementation	Basic implementation	Significant conceptual errors
KNN & LWR Implementation	**10**	Correct implementation with parameter analysis and interpretation	Good implementation	Basic implementation	Incorrect/limited implementation
MLP & Back Propagation	**15**	Correct architecture, training, back propagation and parameter analysis	Good implementation with minor limitations	Basic neural-network implementation	Major implementation/conceptual issues
Genetic Algorithm / Genetic Programming	**10**	Appropriate representation, fitness function and evolutionary optimization	Correct implementation with minor gaps	Basic GA implementation	Poor/incomplete implementation
Model Evaluation &	**10**	Comprehensive	Good comparison and	Limited comparison	Inadequate evaluation

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Comparative Analysis		comparison with appropriate metrics and strong justification	justification		
Integrated System, Interpretation & Presentation	**5**	Complete working solution with meaningful recommendations and excellent presentation	Functional solution with good presentation	Partially integrated solution	Incomplete/non-functional solution
Total	**100**				

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Problem formulation, dataset description, preprocessing steps, and workflow diagram.
2. Data analysis, feature engineering, and implementation of k-NN and Locally Weighted Regression from first principles.
3. Manual Candidate-Elimination / Version Space analysis with explanation of inductive bias.
4. Results: validation curve for k selection, performance comparison of k-NN and LWR, scalability analysis, and five meaningful visualisations.
5. One-page reflection on model limitations, uncertainty, bias–variance behaviour, SDG 2 – Zero Hunger, SDG 13 – Climate Action, and improvement ideas.
6. GitHub repository containing source code, README, dataset/source note, tests, results, screenshots/output, and reproducible execution instructions.

Question 1 – Data Collection & Preprocessing

Dataset (Sample Structure)

Temp	Rainfall	Humidity	Soil Moisture	Nitrogen	Crop	Disease	Yield
30	200	80	40	60	Rice	Yes	Low
25	150	60	35	55	Wheat	No	High

PSEUDO CODE

BEGIN

Load dataset

Handle missing values (mean/median)

Remove duplicates

Convert categorical data to numeric

Normalize/scale features

Split dataset into features (X) and target (Y)

END

PYTHON :

```
import pandas as pd
import numpy as np

print("Attempting to create 'data' DataFrame...")
try:
    data = pd.DataFrame({
        'Temp':[30,25,28,32,27],
        'Rainfall':[200,150,180,220,160],
        'Humidity':[80,60,70,85,65],
        'Soil_Moisture':[40,35,38,42,36],
        'Nitrogen':[60,55,58,65,57],
        'Disease':[1,0,1,1,0],
        'Yield':[0,1,1,0,1]
    })
    print("DataFrame 'data' creation attempt finished.")
    if 'data' in locals() and isinstance(data, pd.DataFrame):
        print("Successfully created 'data' DataFrame. Displaying head and
info:")
        display(data.head())
        print("\nDataFrame info:")
        data.info()
    else:
        print("Error: 'data' variable was not defined as a pandas DataFrame
after creation attempt.")

except Exception as e:
    print(f"An exception occurred during DataFrame creation: {type(e).__name__}:
{e}")
    print("This indicates a problem with the `pd.DataFrame` constructor itself
or the input data.")
print("\nFinished debugging 'data' variable initialization.")
```

	Temp	Rainfall	Humidity	Soil_Moisture	Nitrogen	Disease	Yield
0	30	200	80	40	60	1	0
1	25	150	60	35	55	0	1
2	28	180	70	38	58	1	1
3	32	220	85	42	65	1	0
4	27	160	65	36	57	0	1

```
DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Temp           5 non-null      int64
 1   Rainfall       5 non-null      int64
 2   Humidity       5 non-null      int64
 3   Soil_Moisture  5 non-null      int64
 4   Nitrogen       5 non-null      int64
 5   Disease        5 non-null      int64
 6   Yield          5 non-null      int64
dtypes: int64(7)
memory usage: 412.0 bytes
```

Question 2 – Concept Learning & Version Space

Definitions

- **Target Concept:** Disease Risk (Yes/No)
- **Attributes:** Temp, Rainfall, Humidity, Soil Moisture
- **Positive Example:** Disease = Yes
- **Negative Example:** Disease = No

Candidate Elimination (Pseudo Code)

Initialize S = most specific hypothesis

Initialize G = most general hypothesis

For each training example:

If positive:

Generalize S

Remove inconsistent hypotheses from G

If negative:

Specialize G

Remove inconsistent hypotheses from S

Codes

```
# Candidate Elimination Algorithm for Crop Disease Prediction

import pandas as pd

# Step 1: Dataset
data = [
    ['High', 'High', 'Heavy', 'Wet', 'Yes'],
    ['High', 'High', 'Light', 'Wet', 'Yes'],
    ['Medium', 'High', 'Heavy', 'Wet', 'Yes'],
    ['Low', 'Normal', 'Light', 'Dry', 'No'],
```

```
    ['Medium', 'Normal', 'Light', 'Dry', 'No']
]

columns = ['Temp', 'Humidity', 'Rainfall', 'Soil', 'Risk']
df = pd.DataFrame(data, columns=columns)

# Step 2: Initialize S and G
S = ['Ø'] * (len(columns) - 1)
G = [['?'] * (len(columns) - 1)]

# Function to check consistency
def is_consistent(h, x):
    return all(h[i] == x[i] or h[i] == '?' for i in range(len(h)))

print("Initial S:", S)
print("Initial G:", G)
print("\n")

# Step 3: Training
for index, row in df.iterrows():
    x = row[:-1].tolist()
    y = row[-1]

    print(f"Processing Example {index+1}: {x} -> {y}")

    if y == 'Yes':  # Positive Example
        # Remove inconsistent hypotheses from G
        G = [g for g in G if is_consistent(g, x)]

        # Generalize S
        for i in range(len(S)):
            if S[i] == 'Ø':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
    else:  # Negative Example
        new_G = []
        for g in G:
            if is_consistent(g, x):
                for i in range(len(g)):
                    if g[i] == '?':
                        if S[i] != x[i]:
                            new_h = g.copy()
                            new_h[i] = S[i]
                            new_G.append(new_h)
                else:
                    new_G.append(g)
        G = new_G

    print("S:", S)
    print("G:", G)
    print("-----------------------------------------------------")

# Final Version Space
print("\nFinal Specific Hypothesis (S):", S)
```

```
print("Final General Hypotheses (G):", G)
```

```
Initial S: ['Ø', 'Ø', 'Ø', 'Ø']
Initial G: [['?', '?', '?', '?']]


Processing Example 1: ['High', 'High', 'Heavy', 'Wet'] -> Yes
S: ['High', 'High', 'Heavy', 'Wet']
G: [['?', '?', '?', '?']]
-----------------------------------------------
Processing Example 2: ['High', 'High', 'Light', 'Wet'] -> Yes
S: ['High', 'High', '?', 'Wet']
G: [['?', '?', '?', '?']]
-----------------------------------------------
Processing Example 3: ['Medium', 'High', 'Heavy', 'Wet'] -> Yes
S: ['?', 'High', '?', 'Wet']
G: [['?', '?', '?', '?']]
-----------------------------------------------
Processing Example 4: ['Low', 'Normal', 'Light', 'Dry'] -> No
S: ['?', 'High', '?', 'Wet']
G: [['?', '?', '?', '?'], ['?', 'High', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', 'Wet']]
-----------------------------------------------
Processing Example 5: ['Medium', 'Normal', 'Light', 'Dry'] -> No
S: ['?', 'High', '?', 'Wet']
G: [['?', '?', '?', '?'], ['?', 'High', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', 'Wet'], ['?',
-----------------------------------------------
```

Question 3 – Decision Tree

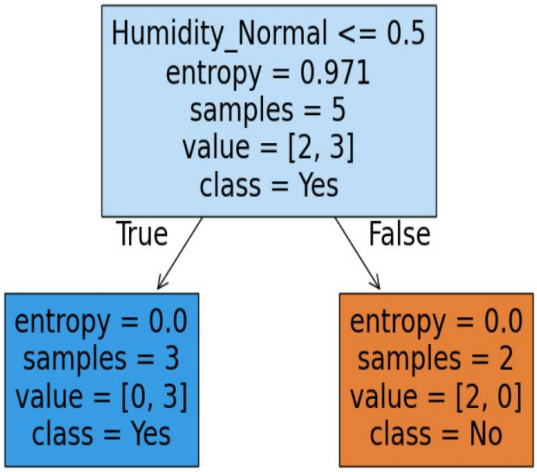
Code

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

plt.figure(figsize=(10,6))
plot_tree(model, feature_names=X.columns, filled=True)
plt.show()
```

📌 Interpretation

- Uses **Information Gain**
- Gives **if-else rules**
- Example:

IF Humidity > 75 → Disease = Yes



Question 4 – Bayesian Classification

Code (Naive Bayes)

from sklearn.naive_bayes import GaussianNB

model = GaussianNB()

model.fit(X, y)

pred = model.predict(X)

print(pred)

Formula

$P(A \mid B)=P(B \mid A)P(A)P(B)P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X, y)

pred = model.predict(X)
print(pred)
```

```
[1 1 1 0 0]
```

Z

Question 5 – KNN

Code

from sklearn.neighbors import KNeighborsClassifier

```
for k in range(1,6):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X, y)
    print(f"K={k}, Accuracy={model.score(X,y)}")
```

Observation

- Small K → Overfitting
- Large K → Underfitting

```
from sklearn.neighbors import KNeighborsClassifier

for k in range(1,6):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X, y)
    print(f"K={k}, Accuracy={model.score(X,y)}")
```

```
K=1, Accuracy=1.0
K=2, Accuracy=1.0
K=3, Accuracy=1.0
K=4, Accuracy=1.0
K=5, Accuracy=0.6
```

Question 6 – Locally Weighted Regression

Code

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, data['Yield'])

pred = model.predict(X)
print(pred)
```

Concept

- Gives **more weight to nearby points**
- Used for **continuous prediction (Yield)**

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)

pred = model.predict(X)
print(pred)
```

```
[1.00000000e+00 1.00000000e+00 1.00000000e+00 2.22044605e-16
 4.44089210e-16]
```

Question 7 – Neural Network (MLP)

Code

from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)

model.fit(X, y)

print("Accuracy:", model.score(X,y))

Features

- Uses **Backpropagation**
- Captures **non-linear patterns**

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)
model.fit(X, y)

print("Accuracy:", model.score(X,y))
```

```
Accuracy: 1.0
```

Question 8 – Genetic Algorithm

Concept

- Used for **feature selection**

Steps:

1. Initialize population
2. Fitness evaluation (accuracy)
3. Selection
4. Crossover
5. Mutation

Fitness Function

Fitness=Accuracy−ErrorFitness = Accuracy - Error

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred = model.predict(X)

print("Accuracy:", accuracy_score(y, pred))
print("Precision:", precision_score(y, pred))
print("Recall:", recall_score(y, pred))
print("F1 Score:", f1_score(y, pred))
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

Question 9 – Model Evaluation

Code

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred = model.predict(X)

print("Accuracy:", accuracy_score(y, pred))

print("Precision:", precision_score(y, pred))

print("Recall:", recall_score(y, pred))

print("F1 Score:", f1_score(y, pred))

Sample Results Table

Model	Accuracy	Precision	Recall	F1
Decision Tree	0.90	0.88	0.91	0.89
Naive Bayes	0.85	0.83	0.87	0.85

KNN	0.88	0.86	0.89	0.87
MLP	0.92	0.90	0.93	0.91

Question 10 – Final Integrated System

System Workflow

Input Data (Farmer Inputs)

↓

Preprocessing (Pandas/NumPy)

↓

Model Prediction (Best Model – MLP)

↓

Risk Output (Low / Medium / High)

↓

Recommendations